Practical:
From Communication to Messaging

LIAU, Yee Yeng Ph.D.

2026

Fall Semester

Contents

Python Network Programming → MQTT → Edge Action



- **Goal:** Observe how **low-level transport communication** becomes **application-level IoT messaging and action**

Practical Flow

Activity	Main output
Networking basics	IP, port, socket, TCP/UDP socket types
TCP socket programming	Client/server exchange JSON
UDP socket programming	Datagram send/receive + receiver-off experiment
TCP vs UDP interpretation	Observation table
Transition to MQTT	Why application protocols are needed
MQTT publisher/subscriber	Brokered sensor data messaging
Edge processor + actuator	MQTT command generated from sensor data
Node-RED demo + submission	Visual flow + evidence collection

Setup

Create practical folder

network-mqtt/

- tcp_server.py
- tcp_client.py
- udp_receiver.py
- udp_sender.py
- mqtt_subscriber.py
- mqtt_sensor.py
- edge_processor.py
- mqtt_actuator.py

Networking Basics

Three concepts

Concept	Meaning in this practical	Example
IP address	Identifies the host	127.0.0.1 = localhost
Port	Identifies the application/process	5000, 5001, 1883
Socket	Programming interface to transport communication	TCP or UDP socket

```
HOST = “127.0.0.1” # localhost on computer – standard IPv4 loopback address → this same computer
PORT = 5000          # application endpoint
```

If Mosquitto is running on own PC on `localhost port = 1883,`

→ `127.0.0.1:1883 # connect to MQTT broker on same PC, port 1883`

LAN IP address identifies a device on the local network, for example:

→ `192.168.0.25`

→ `192.168.0.25:1883` → Connect to the MQTT broker running on the computer whose LAN IP is `192.168.0.25`

Networking Basics

Python socket types: TCP vs UDP

```
import socket
# tcp socket
tcp_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)
# udp socket
udp_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_DGRAM
)
```

`socket.socket()`

1. What kind of network address will be used?

   – IPv4 or IPv6

2. What kind of transport communication will be used?

   – TCP or UDP

Conceptually: `socket.socket(address_family, socket_type)`

`socket.socket(socket.AF_INET, ...)`

→This socket will **communicate using IPv4 addresses**, e.g., `127.0.0.1, 192.168.0.10`

→**AF** → Address Family

→**INET** → internet protocol family

→So, AF_INET → Address Family: Internet → IPv4

→If using IPv6, then `socket.AF_INET6`

Networking Basics

Python socket types: TCP vs UDP

```
import socket
# tcp socket
tcp_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)
```

Create an IPv4 TCP socket

AF_INET → IPv4

SOCK_STREAM → TCP

Together:

IPv4 + TCP

TCP → stream protocol

- → Sees data as a continuous stream of bytes
- → E.g, send "`Hello`", then "`World`"
- → TCP treats data as byte stream: "`HelloWorld`"

```
# udp socket
udp_socket = socket.socket(
    socket.AF_INET,
    socket.SOCK_DGRAM
)
```

Create an IPv4 UDP socket

AF_INET → IPv4

SOCK_DGRAM → UDP

Together:

IPv4 + UDP

UDP → individual packet/message

- → E.g, send "`Message A`", then "`Message B`"
- → UDP keeps them as separate datagrams:
  - → Datagram 1 → "`Message A`"
  - → Datagram 2 → "`Message B`"

Unlike TCP, UDP does not guarantee that:

- ➢ Packet arrives
- ➢ Packets arrive in order
- ➢ Packets are not duplicated

Networking Basics

Python socket types: TCP vs UDP

TCP example

- TCP client typically looks like:

```
import socket

client = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

client.connect(("127.0.0.1", 5000))

client.send(b"Hello") # byte object

client.close()
```

- 127.0.0.1 = destination IP
- 5000 = destination port
- Socket transmit byte, not string

1. Create a TCP socket

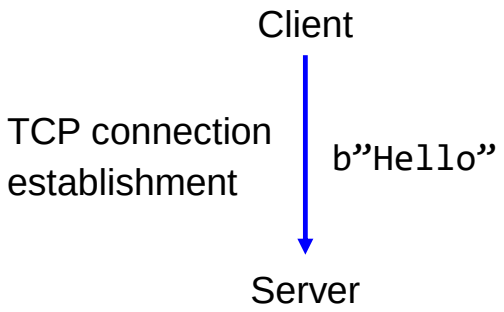
   ```
   client = socket.socket(...)
   ```

2. Connect the socket to the server

   ```
   client.connect(("127.0.0.1", 5000))
   ```

3. Send data through the connection

   ```
   client.send(b"Hello")
   ```



Networking Basics

Python socket types: TCP vs UDP



- A socket is the **software endpoint**
- When we create the socket, we choose whether it uses TCP or UDP
- **TCP / UDP** – transport protocol – how data is sent & received over network
- Simplified analogy:
  - TCP – sealed courier delivery → trackable, missing data resent
  - UDP – postcard → lightweight, connectionless

Temperature sensor
↓
IoT device
↓
Python program
↓
Socket
↓
Network
↓
Server



Networking Basics

Python socket types: TCP vs UDP

UDP example

- UDP client typically looks like:

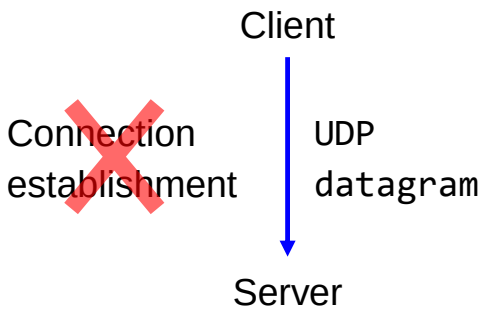
```
import socket

client = socket.socket(
    socket.AF_INET,
    socket.SOCK_DGRAM
)

client.sendto(
    b"Hello",
    ("127.0.0.1", 5000)
)

client.close()
```

Python argument	Networking concept	Layer
AF_INET	IPv4	Network layer
SOCK_STREAM	TCP	Transport layer
SOCK_DGRAM	UDP	Transport layer

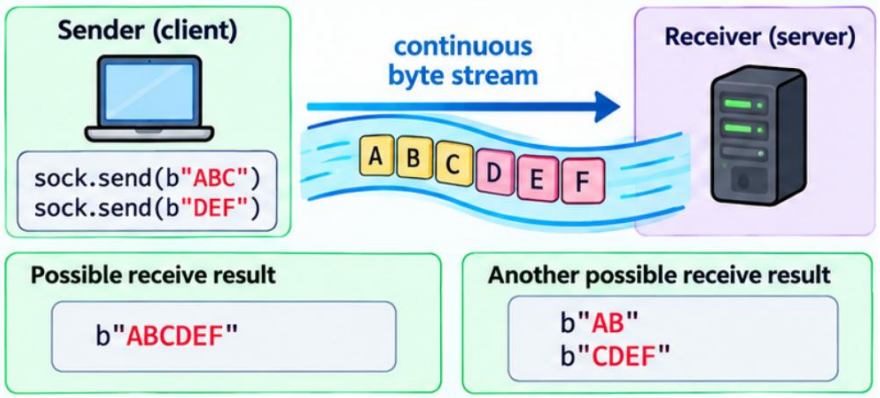


Networking Basics

Python socket types: TCP vs UDP

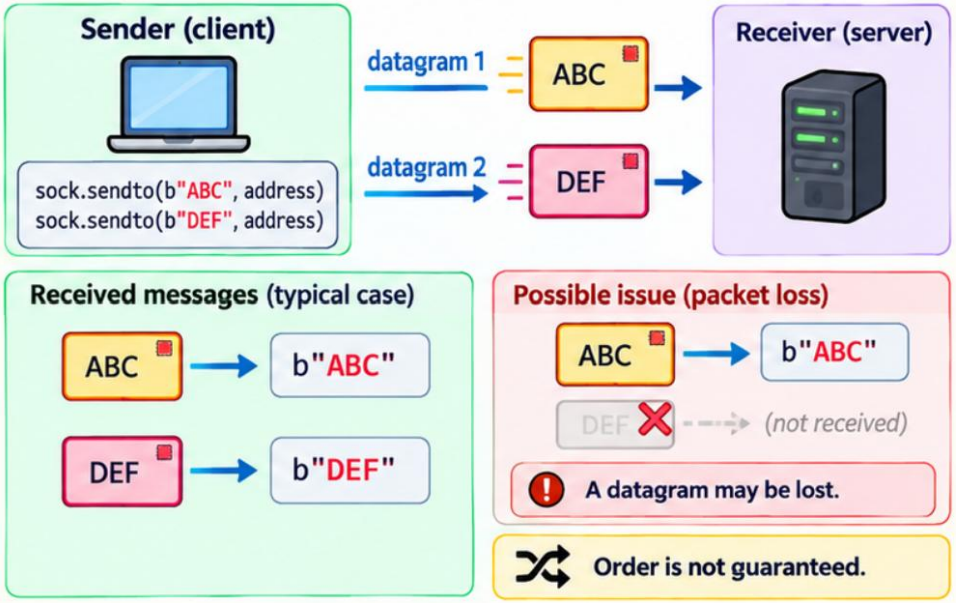
TCP = STREAM

- TCP sends a continuous byte stream
- Does not preserve boundaries of individual `send()` calls



UDP = DATAGRAM

- Each `sendto()` creates a separate datagram
- Message boundaries are preserved



Part 1 – TCP Client/Server

Overview

- **Goal:**
  1. Establish connection
  2. Send JSON
  3. Receive response



- Server waits using listen() and accept()
- Client must connect() before sending data
- Payload is JSON encoded as bytes
- Server receives (recv()) and sends an application-level response

Part 1 – TCP Client/Server

tcp_server.py – Setup & listening

```python
import socket
import json

HOST = "127.0.0.1"
PORT = 5000

# create a TCP socket
server = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

# Attach socket to IP address and port
server.bind ((HOST, PORT))

# start listening for TCP connections
server.listen()

print("TCP server started")
print(f"Waiting at {HOST}: {PORT} ...")
```

- **Server** creates a TCP endpoint and waits for a client
- `socket.AF_INET` means IPv4 addressing
- `SOCK_STREAM` creates a TCP socket
- `bind()` reserves the server endpoint
- `listen()` makes the program wait for incoming TCP clients

Part 1 – TCP Client/Server

tcp_server.py – Receive, decode, respond

```
22  # Wait for a client
23  conn, addr = server.accept()
24
25  print("\nClient connected:")
26  print(addr)
27
28  # Receive data
29  data = conn.recv(1024)
30
31  # Convert received bytes to text
32  message = data.decode()
33
34  print("\nRaw message:")
35  print(message)
36
37  # Convert JSON text to Python dictionary
38  sensor_data = json.loads(message)
39
40  print("\nDecoded sensor data:")
41  print("Device:", sensor_data["device"])
42  print("Temperature:", sensor_data["temperature"], "°C")
43
44  # Send an application-level response
45  conn.sendall(b"Sensor data received")
46
47  # Close connection
48  conn.close()
49  server.close()
50
51  print("\nConnection closed")
```

- **After accept(),** the server can receive bytes from one connected client

- `accept()` blocks until a client connects
- `recv(1024)` receives up to 1024 bytes
- `decode()` changes bytes to text
- `json.loads()` changes JSON text to a Python dictionary
- `sendall()` sends an application response back to the client

Part 1 – TCP Client/Server

tcp_client.py – Prepare JSON and connect

```python
import socket
import json

HOST = "127.0.0.1"
PORT = 5000

sensor_data = {
    "device": "machine01",
    "temperature": 24.3
}

# Convert Python dictionary to JSON
message = json.dumps(sensor_data)

# Create TCP socket
client = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

print("Connecting to server...")

# Establish TCP connection
client.connect((HOST, PORT))

print("Connected")
```

- **Client** turns sensor data into JSON, then establishes the TCP connection

- `sensor_data` is the application data
- `json.dumps()` serializes the dictionary into a JSON string
- `connect()` is the practical evidence that TCP is connection-oriented
- if no server is listening, `connect()` fails

Part 1 – TCP Client/Server

tcp_client.py – Send and receive

```
28  # Convert JSON string to bytes and send
29  client.sendall(message.encode())
30
31  print("Sensor data sent:")
32  print(message)
33
34  # Wait for server response
35  response = client.recv(1024)
36
37  print("\nServer response:")
38  print(response.decode())
39
40  client.close()
```

- **Client** turns sensor data into JSON, then establishes the TCP connection

- `message` is a JSON string
- `encode()` converts text to bytes
- `sendall()` sends bytes over the TCP connection
- `recv()` waits for response bytes from the server
- `decode()` turns the server response back into text

- **Run order: At different terminals:**

1. python tcp_server.py
2. python tcp_client.py

Observation question:

What happens if the client runs before the server is listening?

Part 1 – TCP Client/Server

TCP run: expected output

```
# TCP server
TCP server started
Waiting at 127.0.0.1:5000 ...

Client connected:
('127.0.0.1', 5xxxx)

Raw message:
{"device": "machine01", "temperature": 24.3, "unit": "C"}

Decoded sensor data:
Device: machine01
Temperature: 24.3 °C

Connection closed
```

```
# TCP client
Connecting to server...
Connected

Sensor data sent:
{"device": "machine01", "temperature": 24.3, "unit": "C"}

Server response:
Sensor data received
```

Response sent by server to client after server received data

Part 1 – TCP Client/Server

TCP experiment: If server stopped

- **Steps:**

1. stop tcp_server.py
2. run python tcp_client.py again
3. observe the connection error

Observation question:

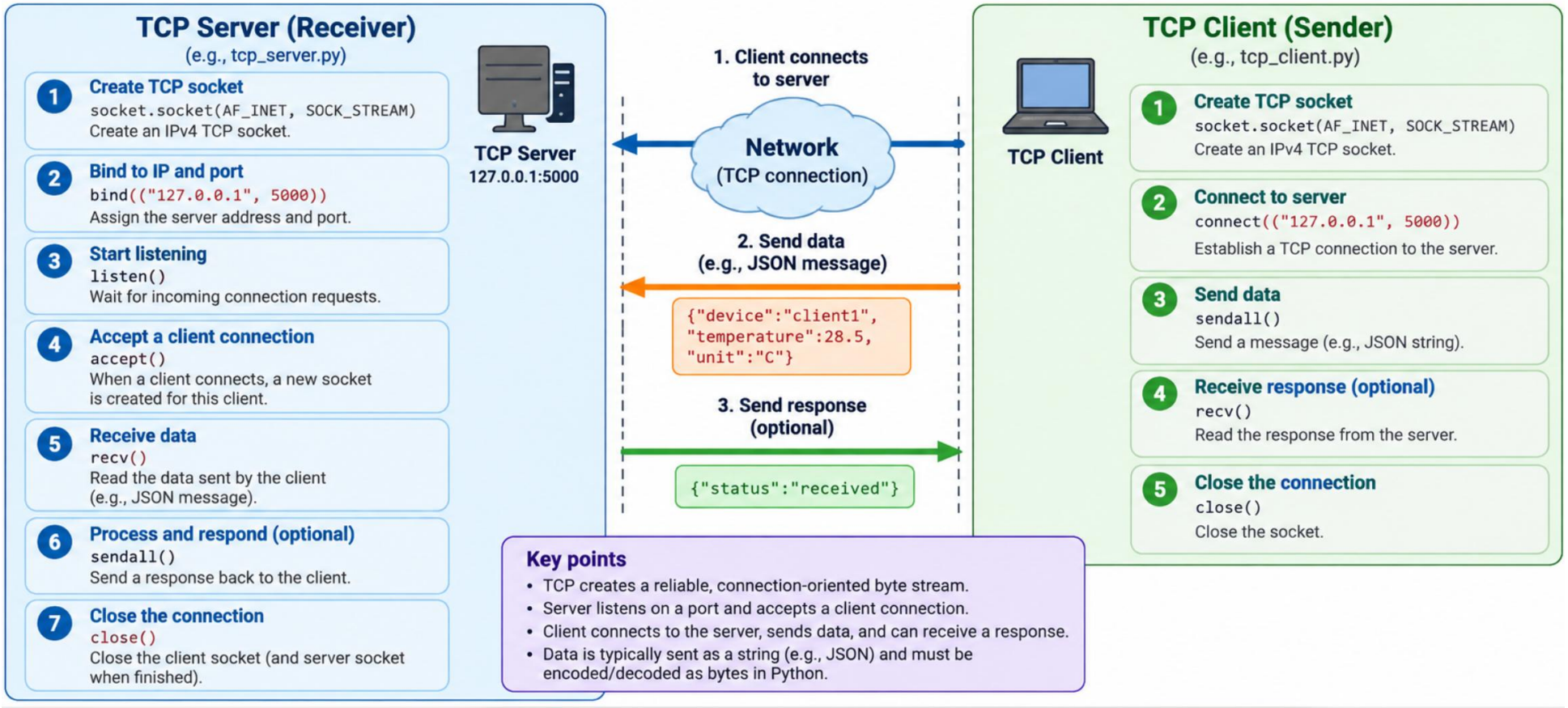
What happens if the client runs before the server is listening?

Expected explanation

- ✓ TCP is connection-oriented
- ✓ The client cannot exchange application data unless a listening server accepts the connection

Part 1 – TCP Client/Server

Complete flow



Part 2 – UDP sender/receiver

Overview

- **Goal:**
  1. Send the same JSON data as a UDP datagram



- No connect() call is needed
- Sender sends a datagram to IP + port
- Receiver gets one datagram using recvfrom()
- Sender may not know whether an application received the message

- UDP still uses IP addresses and ports
- The difference is delivery behavior, not addressing

Part 2 – UDP sender/receiver

udp_receiver.py – Receive one datagram

```python
import socket
import json

HOST = "127.0.0.1"
PORT = 5001

# create UDP socket
receiver = socket.socket(
    socket.AF_INET,
    socket.SOCK_DGRAM
    )

# attach receiver to IP address and port
receiver.bind((HOST, PORT))

print("UDP receiver started")
print(f"Waiting at {HOST} : {PORT} ...")

# receive one UDP datagram
data, addr = receiver.recvfrom(1024)

print("Received from:", addr)

message = data.decode()
print("\nRaw message:")
print(message)

sensor_data = json.loads(message)
print("\nDecoded sensor data:")
print("Device:", sensor_data["device"])
print("Temperature:", sensor_data["temperature"], "°C")

receiver.close()
```

- **Receiver** binds to a UDP port and waits for one datagram

- `SOCK_DGRAM` creates a UDP socket
- `bind()` gives the receiver a local endpoint
- `recvfrom()` returns data and sender address
- There is no `accept()` because UDP has no connection setup

- **UDP receiver:**
  - Receives JSON-encoded sensor data as raw bytes
  - Decodes the bytes into a string
  - Then parses the JSON string into a Python dictionary.

Part 2 – UDP sender/receiver

udp_sender.py – Send one datagram

```python
import socket
import json

HOST = "127.0.0.1"
PORT = 5001

sensor_data = {
    "device": "machine01",
    "temperature": 24.3,
    "unit": "C"
}

# convert dictionary to JSON
message = json.dumps(sensor_data)

# Create UDP socket
sender = socket.socket(
    socket.AF_INET,
    socket.SOCK_DGRAM
)

# send datagram directly
sender.sendto(
    message.encode(),
    (HOST, PORT)
)

print("UDP message sent:")
print(message)

sender.close()
```

- **Sender** sends directly to the receiver endpoint

- There is no `connect()` call
- `sendto()` includes the destination endpoint
- UDP sends a datagram directly
- "`message sent`" does not prove the receiving application obtained it

- **Run order:**

1. python udp_receiver.py
2. python udp_sender.py

Part 2 – UDP sender/receiver

UDP experiment: If receiver stopped

❖ **Steps:**

1. Run udp_receiver.Py and udp_sender.Py once normally
2. Stop udp_receiver.Py
3. Run udp_sender.Py again
4. Observe that sender may still print “UDP message sent”

Observation question:

Did the receiver application receive anything?

Does the sender automatically know?

```
# possible sender output
UDP message sent:
{"device": "machine01", "temperature": 24.3, "unit": "C"}
```

Expected explanation

✓ No. UDP does not establish a connection and does not provide built-in delivery confirmation

TCP vs UDP: Practical observations

Checkpoints

Observation	TCP	UDP
Connection setup	Required	Not required
Main send function	sendall()	sendto()
Main receive function	recv()	recvfrom()
Uses IP + port	yes	yes
Built-in reliability/order	yes	No built-in guarantee
Data model	Byte stream	Datagram

Key lesson:

✓ The payload can be the same JSON data; the transport behavior changes

TCP vs UDP: Practical observations

Same IoT data, different transport behavior




```
sensor_data = {
    "device": "machine01",
    "temperature": 24.3,
    "unit": "C"
}

payload = json.dumps(sensor_data)
```

What changed?

- ✓ Sensor meaning did not change
- ✓ JSON data representation did not change
- ✓ Transport mechanism changed
- ✓ TCP and UDP provide different communication behavior

- ❖ TCP/UDP only move bytes
- ❖ They do not define “publish”, “subscribe”, “topic”, or “device command”

From Sockets to Application Protocols

Needs of MQTT



Raw socket programming gives us:

- ✓ Direct control over TCP/UDP communication
- ✓ IP + port endpoints
- ✓ Bytes sent and received by applications

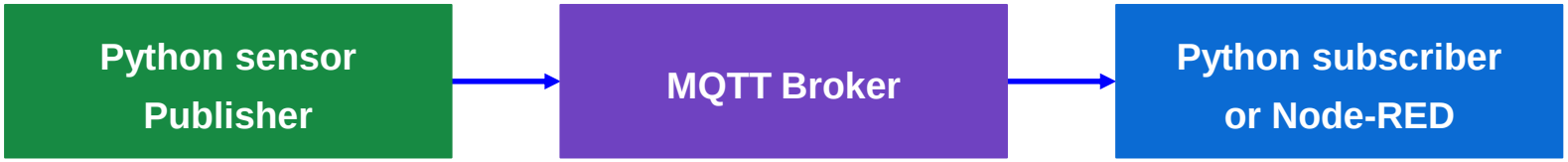
MQTT adds IoT messaging concepts:

- ✓ Publisher and subscriber roles
- ✓ Broker-based message distribution
- ✓ Topics and wildcard subscriptions
- ✓ QoS and retained messages

- ❖ MQTT typically runs over TCP/IP
- ❖ MQTT does not replace TCP; it **uses TCP** and **adds application-level messaging**

Part 3 – MQTT Publisher/Subscriber

Overview



Publisher knows:

- ✓ broker address
- ✓ Topic
- ✓ Payload
- ✓ QoS / retained options

Publisher does not need to know:

- ✓ Subscriber IP address
- ✓ How many subscribers exist
- ✓ Whether the dashboard, database or Node-RED receives the message

This is the architectural benefit:

- ❖ Producer and consumer are **decoupled** through the broker

Setup

Installation

Required software:

- Python 3.x installed
- paho-mqtt installed: `pip install paho-mqtt`
- Mosquitto MQTT broker installed and running
- Node-RED installed for the final visual flow demo

Setup:

- Use 127.0.0.1 for TCP, UDP, MQTT broker and Node-RED
- Use separate terminals for server, client, broker, sensor and actuator

Setup

Mosquitto MQTT Broker Setup

❖ Use Mosquitto as the broker; use Paho MQTT inside Python for publish/subscribe clients

Setup sequence:

1. Install Mosquitto
   - https://mosquitto.org/download/
   - Default: C:\Program Files\mosquitto
2. Add to Windows path
   1. Click **Windows key** → search: *Environment Variables*
   2. Open: *Edit the system environment variables*
   3. In **System Properties** window, click: *Environment Variables …*
   4. Under **User variables for USER**, select: **Path**, then click **Edit**
   5. Click **New** and add: C:\Program Files\mosquito
   6. Click **OK** on all windows
   7. Close and reopen VS code, test: `mosquito -v`
3. Test the broker and clients
   - Conceptual flow: Open separate terminals for broker, subscriber, and publisher



MQTT routes messages through the broker; clients do not talk directly to each other.

Setup

Mosquitto MQTT Broker Setup

3. Test the broker and clients

Terminal test

Terminal 1 — broker

```
mosquitto -v
```

Terminal 2 — subscriber

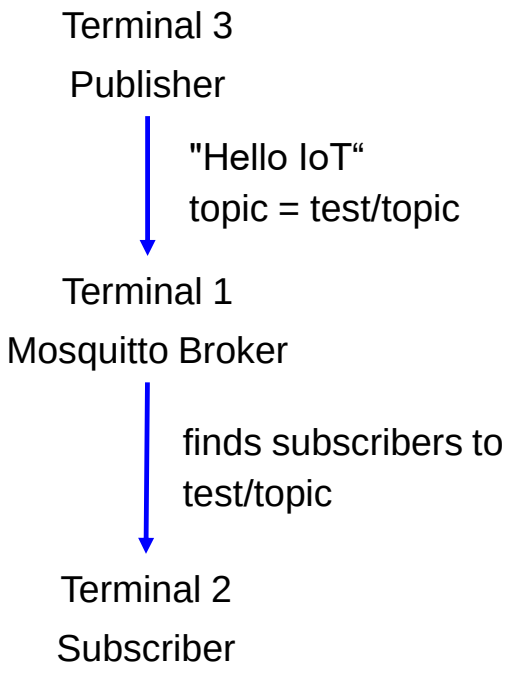
```
mosquitto_sub -h localhost -t "test/topic" -v
```

Terminal 3 — publisher

```
mosquitto_pub -h localhost -t "test/topic" -m "Hello IoT"
```

Expected output in subscriber:

```
test/topic Hello IoT
```



Part	Meaning
-v	Prints connection and message activity
mosquitto_sub	MQTT subscriber client
mosquitto_pub	MQTT publisher client
-h localhost	Connect to the local Mosquitto broker
-t "test/topic"	Publish to this topic
-m "Hello IoT"	Message payload

Part 3 – MQTT Publisher/Subscriber

Start or verify the MQTT broker

Setup:

- ❖ Use paho-mqtt
- ✓ `pip install paho-mqtt`
- ❖ Run broker manually
- ✓ `mosquitto -v`

MQTT default port: 1883

Broker address for all scripts:

- ✓ `BROKER = “127.0.0.1”`
- ✓ `PORT = 1883`

Publisher does not need to know:

- ✓ Mosquitto is the MQTT broker
- ✓ Publisher sends messages to the broker
- ✓ Subscribers receive matching topics from the broker
- ✓ Localhost keeps everything on the same computer

Terminal plan

- ❖ Terminal 1: mosquitto -v
- ❖ Terminal 2: subscriber
- ❖ Terminal 3: sensor publisher
- ❖ Terminal 4: edge processor / actuator

Part 3 – MQTT Publisher/Subscriber

mqtt_subscriber.py – imports and settings

```python
import paho.mqtt.client as mqtt

BROKER = "127.0.0.1"
PORT = 1883

TOPIC = "factory/machine01/temperature"

# called when connection to the broker is established
def on_connect(client, userdata, flags, reason_code, properties):
    print ("Connected to MQTT broker")
    print ("Reason code:", reason_code)

    client.subscribe(TOPIC)

    print("Subscribed to:", TOPIC)
```

1. MQTT configuration

BROKER → Where is the MQTT broker?

PORT → Which broker service?

TOPIC → Which messages do we want?

2. Connection callback

- `client` → our MQTT client
- `reason_code` → did connection succeed?
- `on_connect()` → runs when connection event occurs
- Then, `client.subscribe(TOPIC)`

→ Tell the broker: “Send me messages published to this topic.”

Part 3 – MQTT Publisher/Subscriber

mqtt_subscriber.py – imports and settings

```
# Called whenever a subscribed message is received
def on_message(client, userdata, message):

    print("\nNew message")
    print("Topic:", message.topic)
    print("Payload:", message.payload.decode())
    print("QoS:", message.qos)


# Create MQTT client
client = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2
)

# Assign callback functions
client.on_connect = on_connect
client.on_message = on_message
```

3. Message callback

- Automatically called whenever a matching MQTT message arrive
- `message.topic` → MQTT topic
- `message.payload` → actual application data

4. Create and configure client

- Creates MQTT client
- `VERSION2` - Create an MQTT client using Paho's current callback interface
- Then,

When connection happens

→ run `on_connect()`

When message arrives

→ run `on_message()`

Part 3 – MQTT Publisher/Subscriber

mqtt_subscriber.py – imports and settings

```
# connect to MQTT broker
client.connect(
    BROKER,
    PORT,
    keepalive=60
)

print("Waiting for MQTT messages ...")

# Keep client running and processing MQTT messages
client.loop_forever()
```

Run mqtt_subscriber.py before mqtt_sensor.py

5. Connect and wait

- Establishes communication with the broker
- Keep the MQTT client running so it can process network traffic and react to incoming messages

Connect

↓

Subscribe

↓

Wait

↓

Message arrives

↓

on_message()

↓

Wait again

Part 3 – MQTT Publisher/Subscriber

mqtt_sensor.py – simulated sensor publisher

```python
import paho.mqtt.client as mqtt
import json
import random
import time

BROKER = "127.0.0.1"
PORT = 1883

TOPIC = "factory/machine01/temperature"

client = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2
)

client.connect(
    BROKER,
    PORT,
    keepalive=60
)

# Start network processing
client.loop_start()

print("MQTT temperature sensor started")
print("Publishing to:")
print(TOPIC)
```

- `client` connects to the local broker
- `TOPIC` classifies the message
- `loop_start()` runs MQTT network processing in background
- Publisher still does not know subscriber addresses
- `keepalive = 60`

  MQTT client tells the broker:

  - “If there is no normal MQTT traffic for up to 60 seconds, I will still send a keep-alive message so the connection stays active.”

Part 3 – MQTT Publisher/Subscriber

mqtt_sensor.py – simulated sensor publisher

```
28  try:
29
30      while True:
31
32          # Generate simulated temperature
33          temperature = round(
34              random.uniform(25.0, 35.0),
35              1
36          )
37
38          sensor_data = {
39              "device": "machine01",
40              "temperature": temperature,
41              "unit": "C"
42          }
43
44          # Convert dictionary to JSON
45          payload = json.dumps(sensor_data)
46
47          # Publish MQTT message
48          result = client.publish(
49              TOPIC,
50              payload,
51              qos=0
52          )
53
54          print("\nPublished:")
55          print(payload)
56
57          time.sleep(2)
58
59  except KeyboardInterrupt:
60
61      print("\nStopping sensor...")
62
63  finally:
64
65      client.loop_stop()
66      client.disconnect()
```

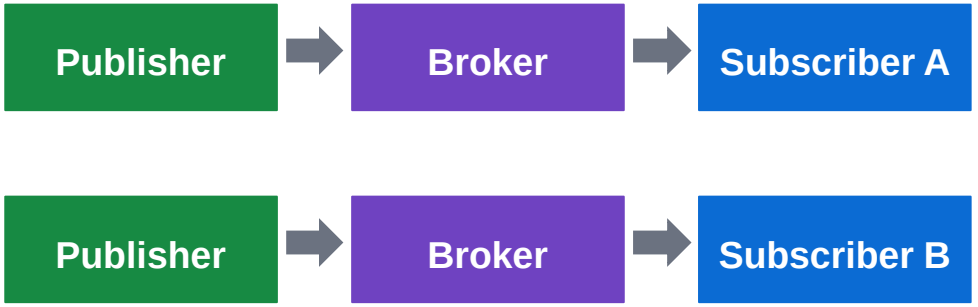
- `random.uniform()` simulates a changing sensor reading
- `json.dumps()` serializes data into a JSON payload
- `client.publish()` sends the MQTT message to the broker
- `qos=0` means no MQTT-level publication acknowledgment/redelivery
- `Ctrl+C` stops the infinite loop

Part 3 – MQTT Publisher/Subscriber

MQTT experiment: multiple subscribers

- **To show producer–consumer decoupling**
- **Steps:**

1. Open two terminals running `mqtt_subscriber.py`
2. Run `mqtt_sensor.py` once
3. Observe that both subscribers receive the same publication
4. Publisher code did not change



Expected explanation

- ✓ The publisher sends to the broker and topic
- ✓ The broker distributes the message to all matching subscribers

Part 3 – MQTT Publisher/Subscriber

MQTT topic wildcard experiment

- **Use one subscription to receive multiple machine temperature topic**
- **Steps:**

1. Copy mqtt_sensor.py to mqtt_sensor_wildcard.py
2. Modify the publisher so that it publishes to several different MQTT topics in each loop
3. Subscriber can test + and #

```
# change subscriber topic
TOPIC = "factory/+/temperature"

# publish topics such as
factory/machine01/temperature
factory/machine02/temperature
factory/machine03/temperature
```

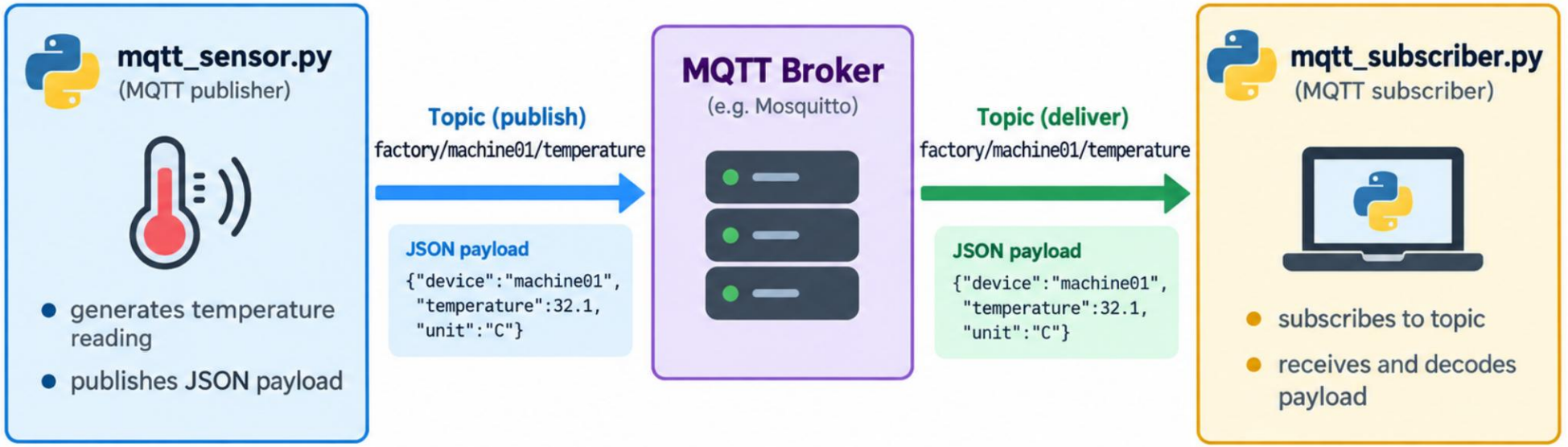
```
# multi-level wildcard
TOPIC = "factory/#"

# matches examples
factory/machine01/temperature
factory/machine01/vibration
factory/machine01/status
```

Wildcard	Meaning	Example
+	exactly one topic level	factory/+/temperature
#	all remaining levels	factory/#

Part 3 – MQTT Publisher/Subscriber

MQTT message flow

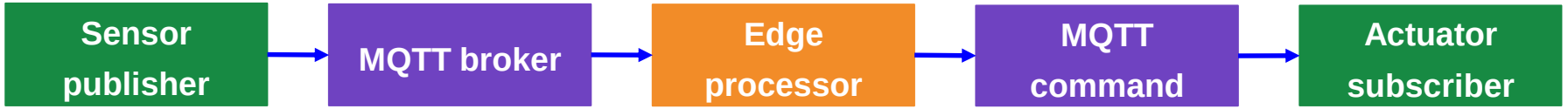


Message Flow Sequence

1. **Sensor publishes message**
   `mqtt_sensor.py` generates a temperature reading and publishes a JSON payload to `factory/machine01/temperature`.
2. **Broker receives message**
   The MQTT broker receives the message on the topic `factory/machine01/temperature`.
3. **Subscriber gets matching topic**
   `mqtt_subscriber.py`, which subscribed to `factory/machine01/temperature`, receives and decodes the JSON payload.

Part 4 – Python Edge Processor

Overview



Edge processing task

- Subscribe to `factory/machine01/temperature`
- Parse JSON payload
- If temperature > 30°C, publish `FAN ON`
- Otherwise, publish `FAN OFF`
- Command topic: `factory/machine01/fan/command`

Part 4 – Python Edge Processor

edge_processor.py – setup and subscribe

```python
import json
import paho.mqtt.client as mqtt

BROKER = "127.0.0.1"
PORT = 1883

TOPIC = "factory/machine01/temperature"
COMMAND_TOPIC = "factory/machine01/fan/command"

def on_connect(client, userdata, flags, reason_code, properties):
    print ("Edge processor connected")

    client.subscribe(TOPIC)

    print("LISTENING to:")
    print(TOPIC)

```

Edge processor is an MQTT subscriber and publisher at the same time

- `SENSOR_TOPIC` receives temperature messages
- `COMMAND_TOPIC` sends fan commands
- edge processor uses the broker like every MQTT client
- `on_connect()` subscribes only after broker connection succeeds

Part 4 – Python Edge Processor

edge_processor.py – setup and subscribe

```python
def on_message(client, userdata, message):

    data = json.loads(
        message.payload.decode()
    )

    temperature = data["temperature"]

    print ("temperature:", temperature, "°C")

    if temperature > 30:
        command = "FAN_ON"
    else:
        command = "FAN_OFF"

    client.publish(
        COMMAND_TOPIC,
        command
    )

    print("Action:", command)

client = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2
)

client.on_connect = on_connect
client.on_message = on_message

client.connect(BROKER, PORT, keepalive=60)

client.loop_forever()
```

Application logic turns sensor data into an actuator command

- `json.loads()` parses the payload into a dictionary
- `temperature` is application data, not MQTT metadata
- `if/else` is the edge decision rule
- `client.publish()` sends the command to another MQTT topic
- This creates a **simple sense–decide–act loop**

Part 4 – Python Edge Processor

mqtt_actuator.py – virtual actuator setup

```python
import paho.mqtt.client as mqtt

BROKER = "127.0.0.1"
PORT = 1883

COMMAND_TOPIC = "factory/machine01/fan/command"

def on_connect(client, userdata, flags, reason_code, properties):

    print("Virtual actuator connected")

    client.subscribe(COMMAND_TOPIC)

    print("Listening for commands:")
    print(COMMAND_TOPIC)

```

Actuator subscribes to the command topic and prints the fan state

- Actuator is an MQTT subscriber
- It subscribes to a command topic, not the sensor topic
- This represents an actuator receiving commands from application logic
- Real hardware could replace the print statements later

Part 4 – Python Edge Processor

mqtt_actuator.py – command handling

```python
def on_message(client, userdata, message):

    command = message.payload.decode()

    print ("\nCommand received:")
    print(command)

    if command == "FAN_ON":
        print("Virtual fan status: ON")
    elif command == "FAN_OFF":
        print("Virtual fan status: OFF")
    else:
        print("Unknown command")

client = mqtt.Client(
    mqtt.CallbackAPIVersion.VERSION2
)

client.on_connect = on_connect
client.on_message = on_message

client.connect(BROKER, PORT, keepalive=60)

client.loop_forever()
```

Virtual actuator changes printed state based on MQTT command payload

- `message.payload` is bytes from MQTT
- `decode()` converts command to text
- The command is application-defined
- `FAN ON / FAN OFF` simulates physical actuation
- This closes the control loop

Part 4 – Python Edge Processor

Run the complete Python MQTT edge loop

Terminal plan

- Terminal 1: mosquitto -v
- Terminal 2: python mqtt_sensor.py
- Terminal 3: python edge_processor,py
- Terminal 4: python mqtt_actuator.py



Observation:

When random temperature rises above 30°C, the command should become FAN ON

```
# actuator output example
Command received:
FAN OFF
Virtual fan status: OFF

Command received:
FAN ON
Virtual fan status: ON
```

Part 4 – Python Edge Processor

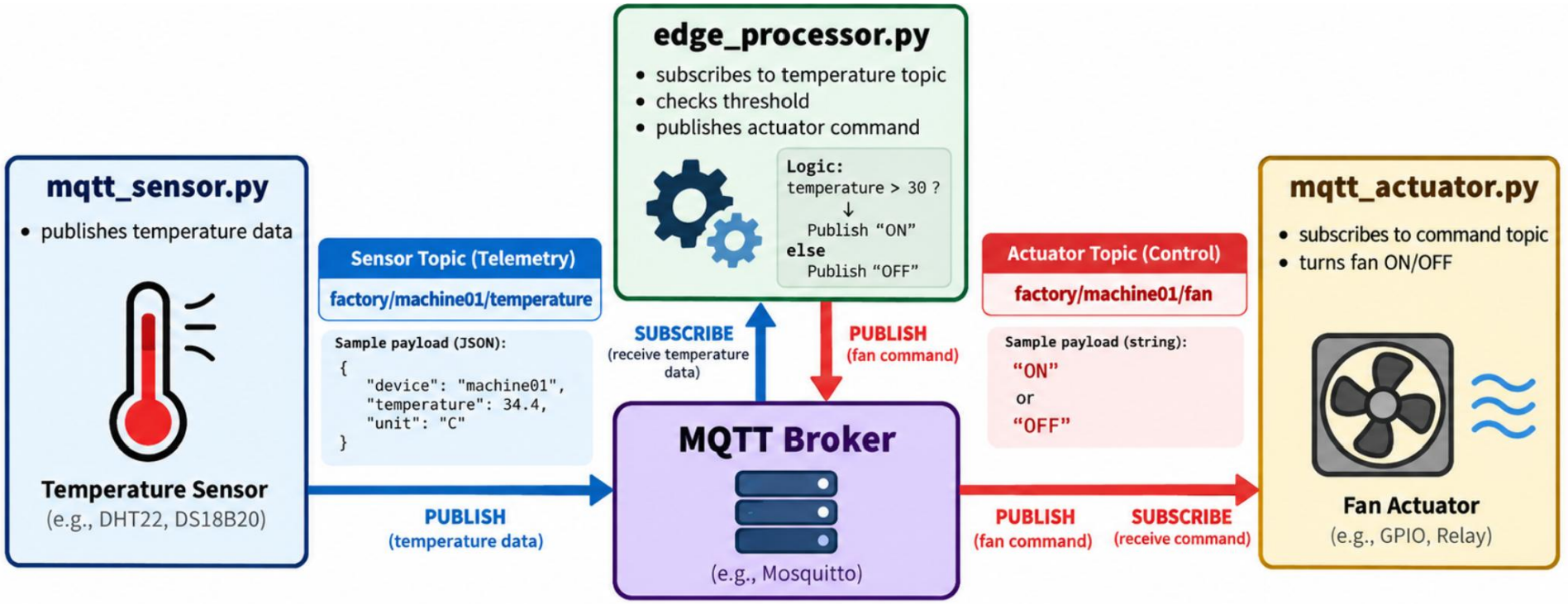
Interpreting the edge loop

Component	Role in IoT system
mqtt_sensor.py	simulated sensing device / publisher
JSON payload	structured data representation
Mosquitto	local MQTT broker / edge communication point
edge_processor.py	local processing and decision logic
mqtt_actuator.py	virtual actuator / subscriber
FAN ON / FAN OFF	application-defined command

Complete simulated IoT loop: **sense → communicate → process → act**

Part 4 – Python Edge Processor

Message flow



Message Flow Summary

1 mqtt_sensor.py
Reads temperature from sensor and publishes to topic
`factory/machine01/temperature`
Payload:
`{"device": "machine01", "temperature":34.4, "unit":"C"}`

2 MQTT Broker
Receives the temperature message and delivers it to the subscriber (edge_processor.py).

3 edge_processor.py
Subscribes to `factory/machine01/temperature`, checks:
`temperature > 30 ?`
If yes, publishes **"ON"** else publishes **"OFF"** to `factory/machine01/fan,`

4 MQTT Broker
Receives the fan command ("ON" or "OFF") and delivers it to the subscriber (mqtt_actuator.py).

5 mqtt_actuator.py
Subscribes to `factory/machine01/fan`, receives the command and turns the fan ON or OFF.

Part 5 — Node-RED Visual Flow Demo

Overview



Use Node-RED to show

- MQTT message input
- JSON parsing
- temperature threshold decision
- virtual action command
- MQTT output to actuator topic

Node-RED is not mainly a network simulator. In this class, it is a visual IoT integration and application-logic tool.

Setup

Install Node-Red

1. Install Node.js LTS from https://nodejs.org/en/download
2. Close and reopen VS code, then verify:
   - node --version
   - npm --version
3. Install Node-RED globally
   - npm install -g --unsafe-perm node-red
4. Start Node-RED:
   - node-red

   ➢ Output indicating server has started, usually at: http://127.0.0.1:1880

   ➢ Open the address in browser

   ➢ If node-red is not recognized after installation, run:

```
npm config get prefix
```

Part 5 — Node-RED Visual Flow Demo

Node-RED flow: required nodes

Node	Purpose	Important setting
MQTT In	subscribe to sensor data	topic: factory/machine01/temperature
JSON	parse JSON text into object	msg.payload becomes object
Debug	show received payload	complete msg.payload
Switch	test temperature threshold	msg.payload.temperature > 30
Change	create command	msg.payload = FAN ON / FAN OFF
MQTT Out	publish command	topic: factory/machine01/fan/command

Part 5 — Node-RED Visual Flow Demo

Node-RED flow: required nodes

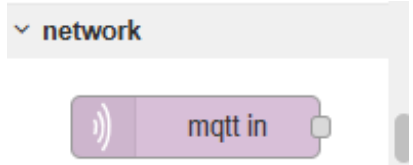
Field	Meaning	Example
msg.topic	**Where / what category the message belongs to**	factory/machine01/temperature
msg.payload	**The actual data carried by the message**	{"temperature":34.4,"unit":"C"}

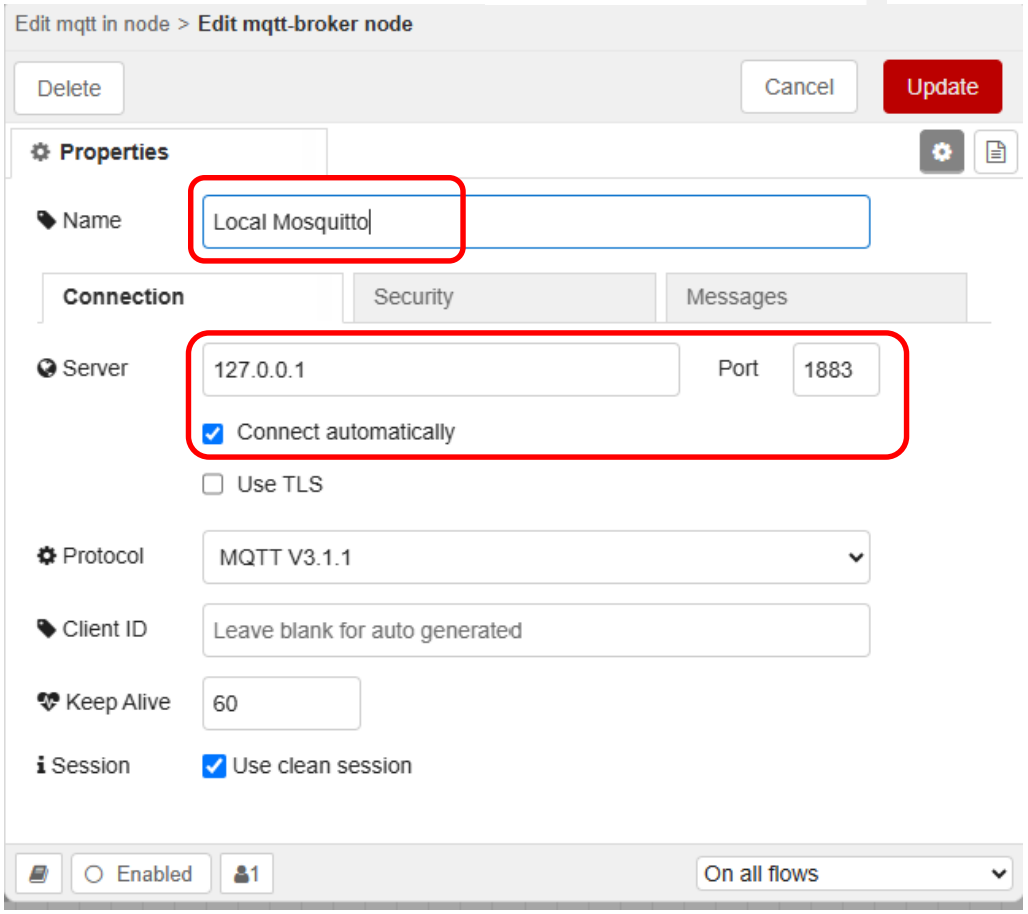
- msg.topic = label / route
- msg.payload = content

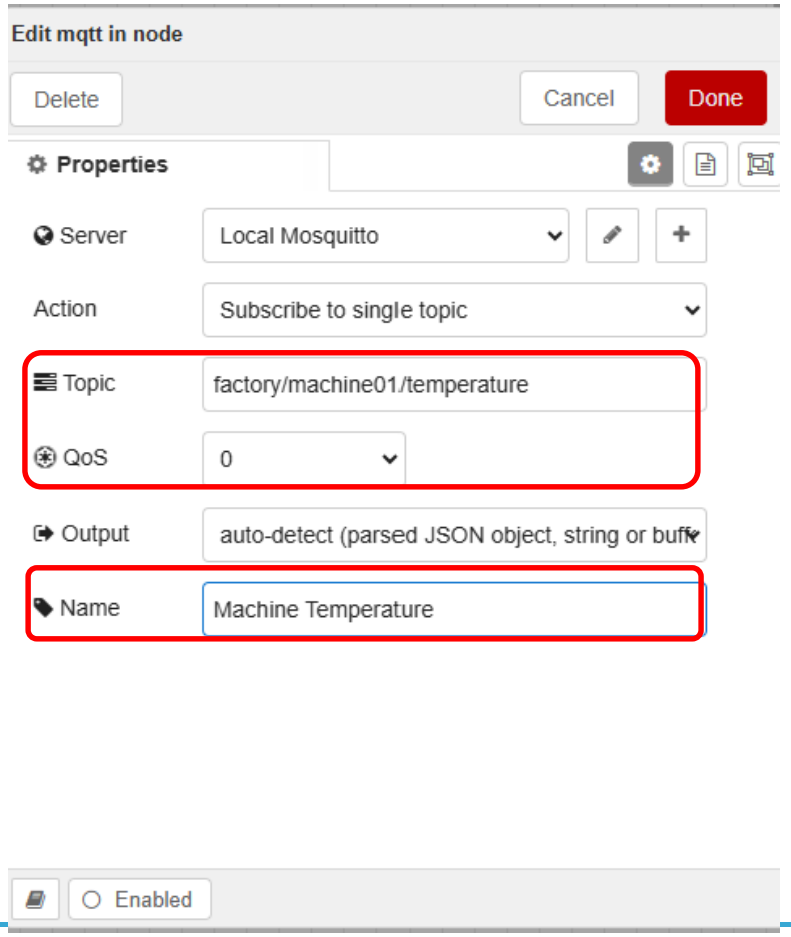
Part 5 — Node-RED Visual Flow Demo

Step 1: MQTT In + JSON + Debug

1. Add mqtt in node



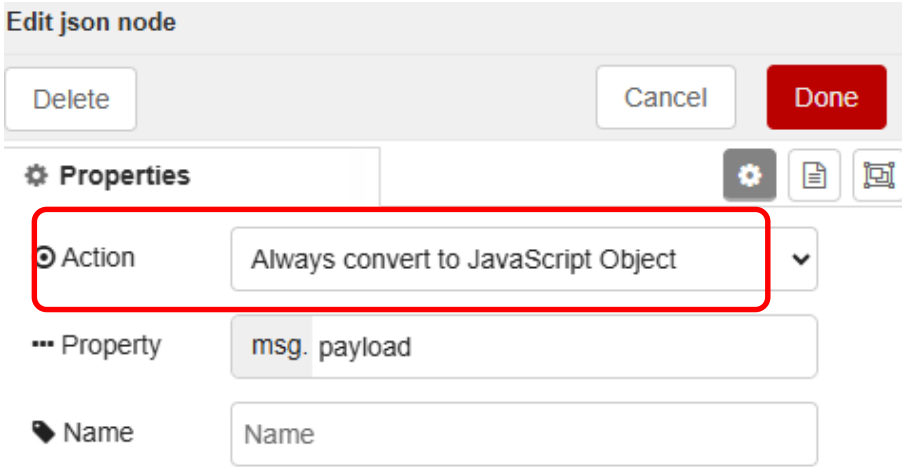




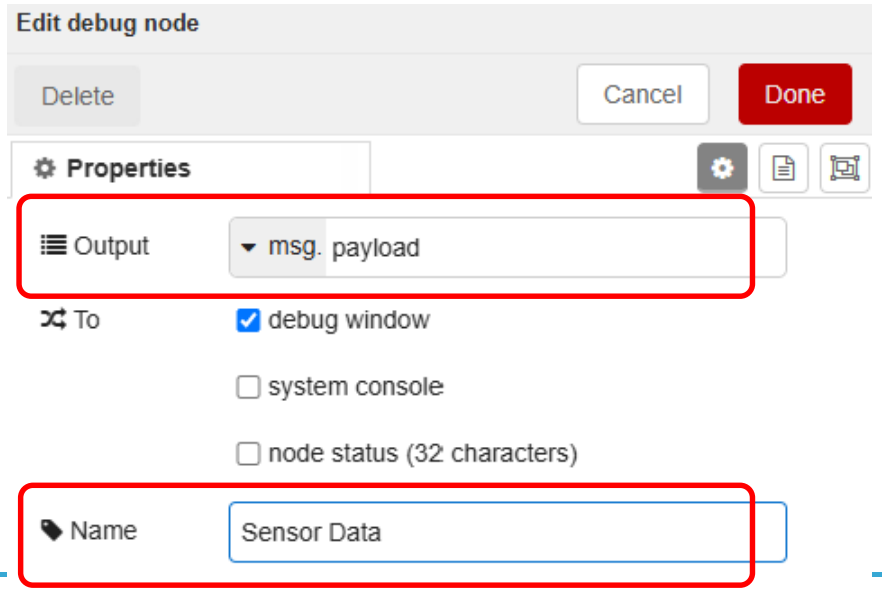
Part 5 — Node-RED Visual Flow Demo

Step 1: MQTT In + JSON + Debug

2. Add json node



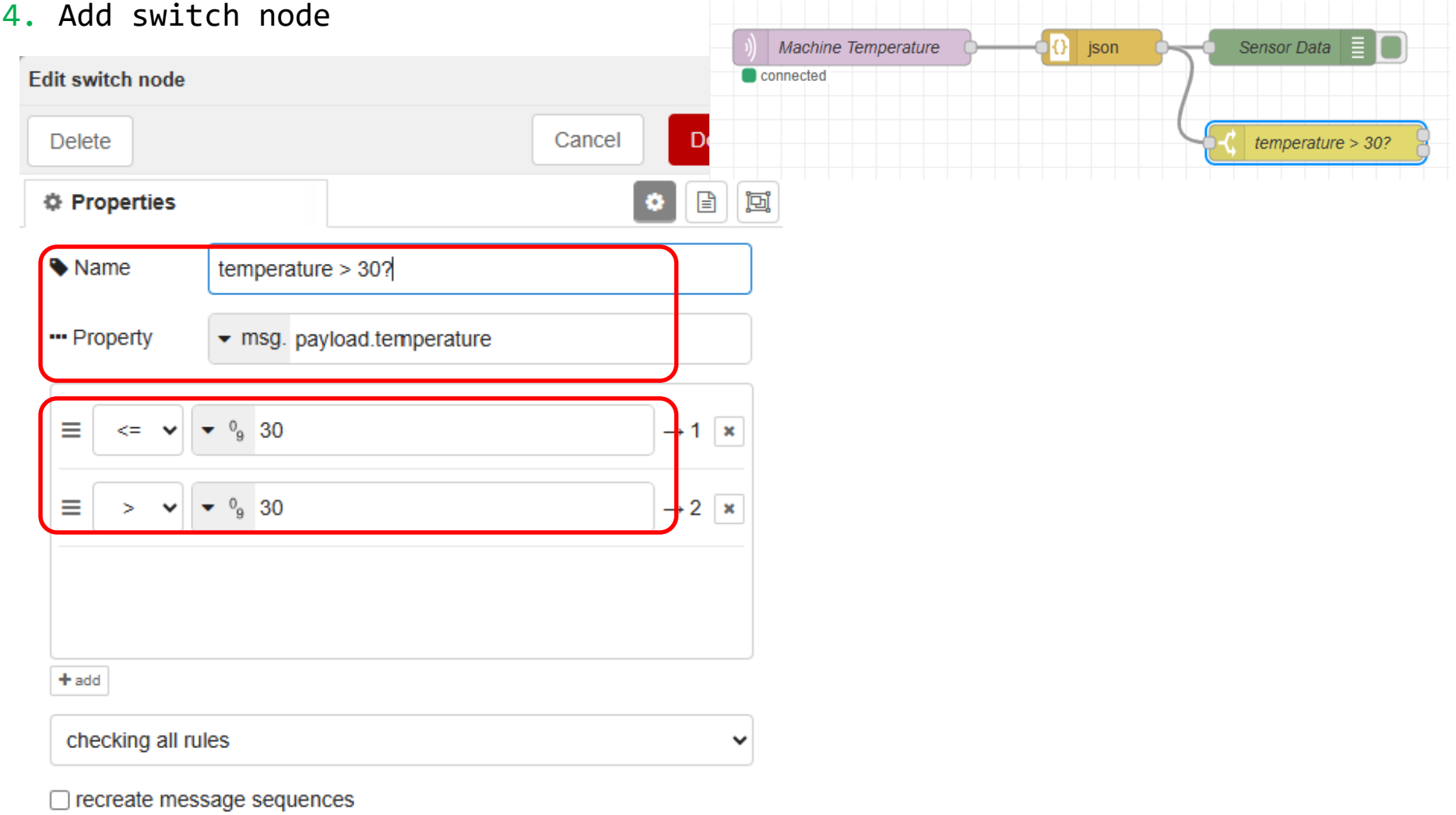
3. Add Debug node



Part 5 — Node-RED Visual Flow Demo

Step 2: Decision logic

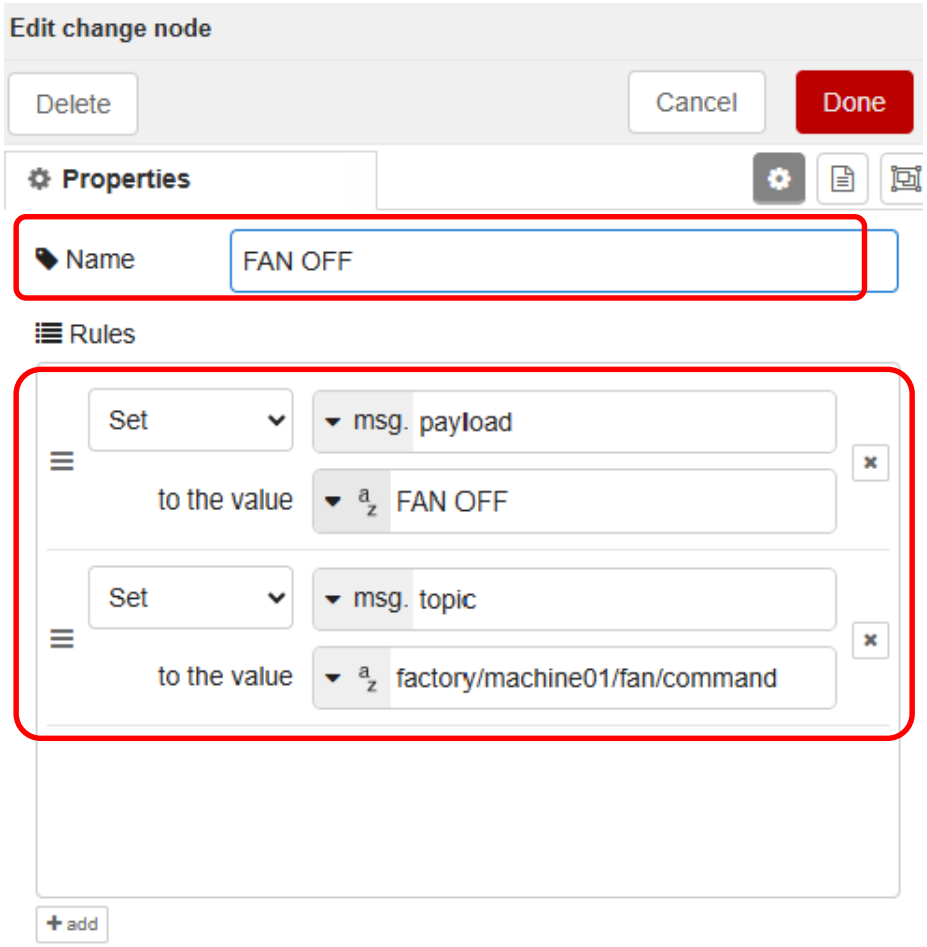
4. Add switch node



Part 5 — Node-RED Visual Flow Demo

Step 2: Decision logic

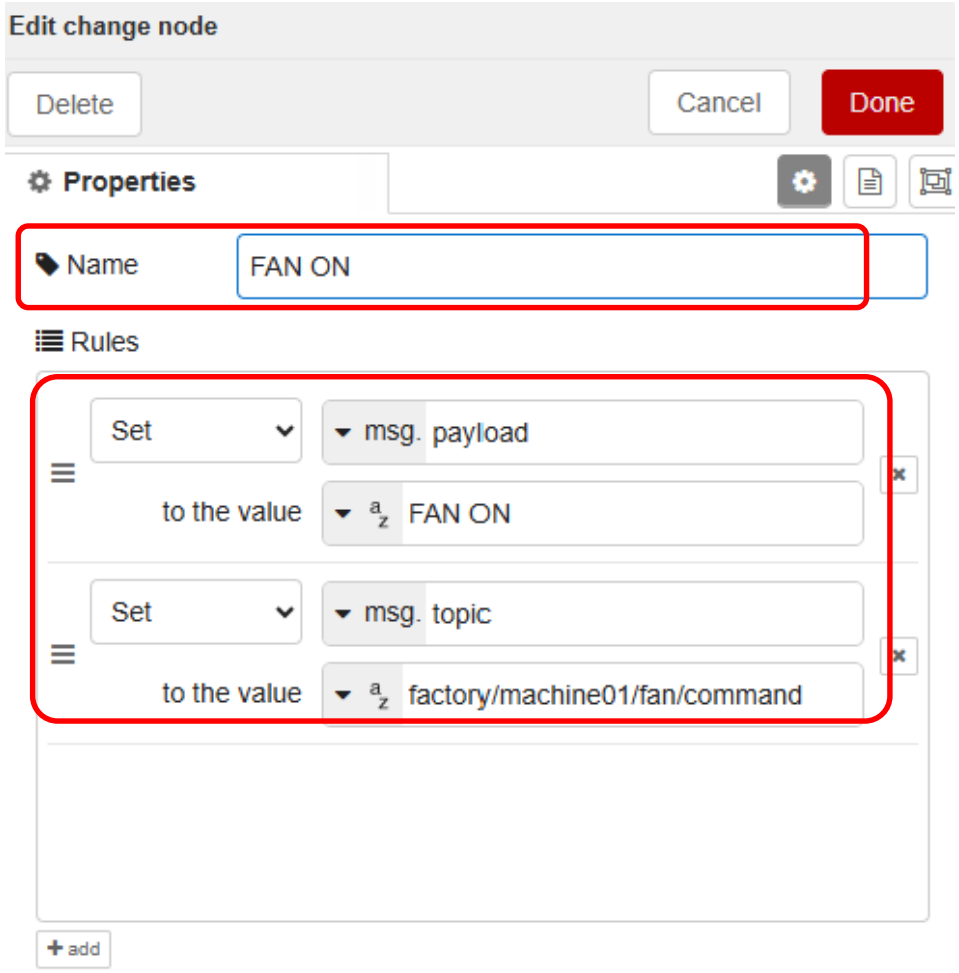
5. Add change node



Part 5 — Node-RED Visual Flow Demo

Step 2: Decision logic

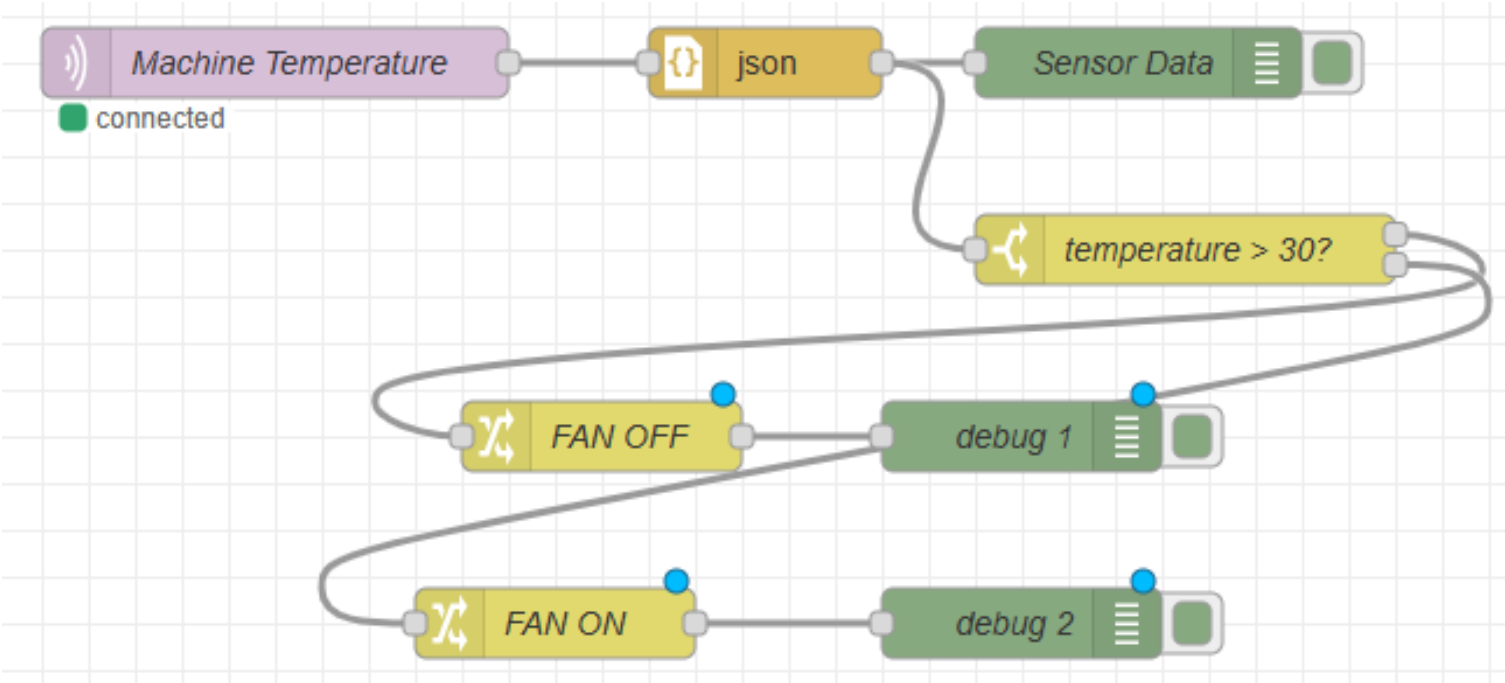
5. Add another change node



Part 5 — Node-RED Visual Flow Demo

Step 2: Decision logic

6. Add two debug nodes, one after FAN OFF, and another after FAN ON



Part 5 — Node-RED Visual Flow Demo

Step 3: MQTT Out to actuator

7. Add mqtt out node:
   - Topic: Leave empty
   - QoS: 0
   - Retain: false
   - Name: Fan command

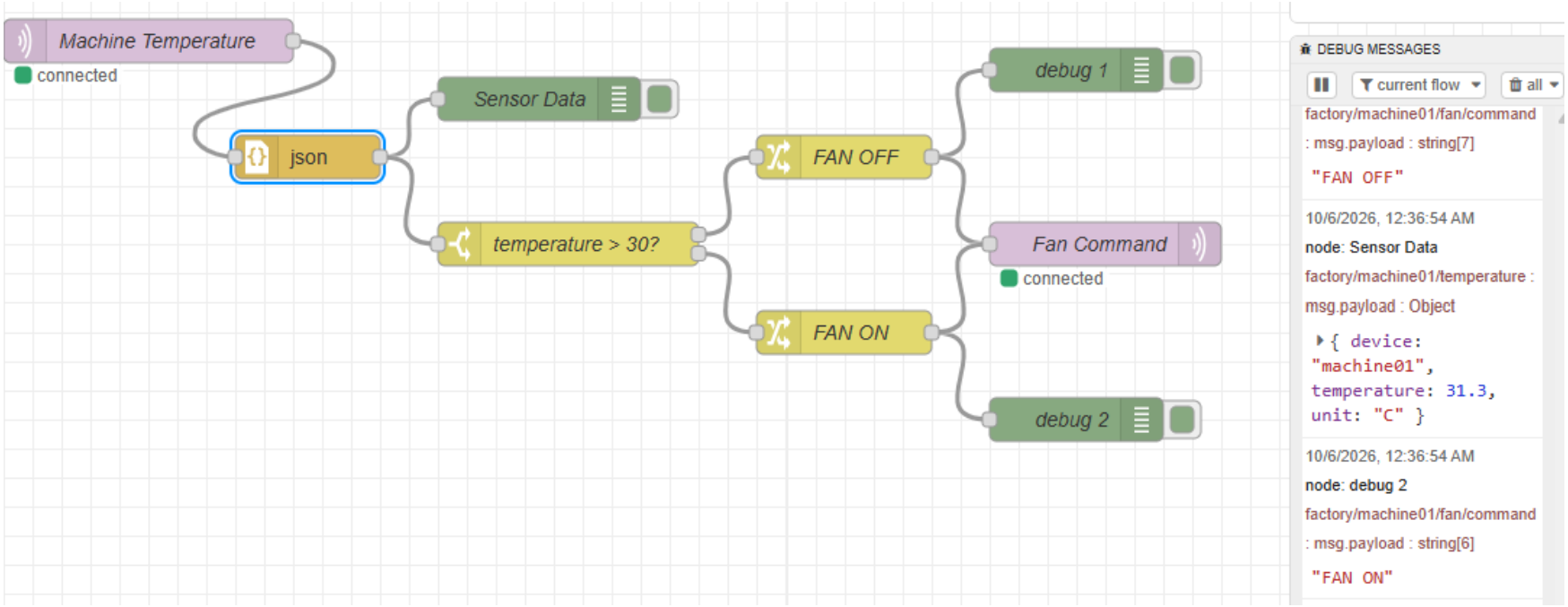
8. Terminals:

Terminal 1: mosquitto -v

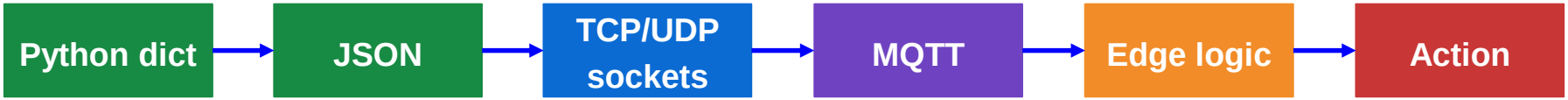
Terminal 2: python mqtt_sensor.py

Browser: Node-RED

Terminal 3: python mqtt_actuator.py



Summary



Practical part	What it demonstrates
TCP sockets	connection-oriented process-to-process communication
UDP sockets	connectionless datagram communication
MQTT	brokered publish–subscribe IoT messaging
Edge processor / Node-RED	local application logic close to the data source
Virtual actuator	command/action side of an IoT loop